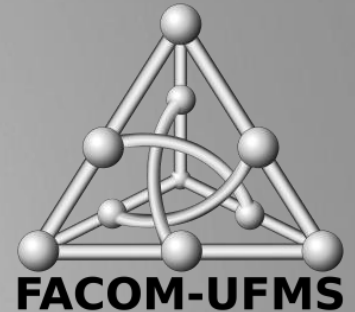




**FUNDAÇÃO
UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL**



GERÊNCIA DE CONFIGURAÇÃO DE SOFTWARE

MÓDULO 2 - ATIVIDADES DE GERÊNCIA DE CONFIGURAÇÃO DE SOFTWARE

Profa. Dra. Jane Sandim Eleutério

jane.eleuterio@ufms.br

O QUE VAMOS APRENDER HOJE?

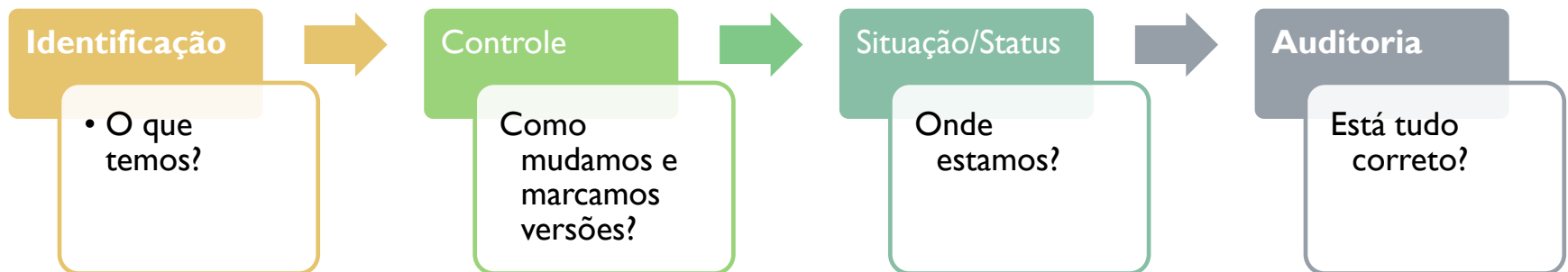


➤ Objetivos do Módulo

- Detalhar as atividades práticas que compõem o processo de gerência de configuração.
- Aprender a identificar e atribuir propriedades aos Itens de Configuração (ICs)
- Compreender o conceito e a importância das Linhas de Base (Baselines)
- Conhecer o papel dos Relatórios de Status na visibilidade do projeto
- Diferenciar Auditoria Física de Auditoria Funcional

RECAPITULAÇÃO: DA TEORIA À PRÁTICA

- No Módulo 1, vimos que a GCS garante a integridade
- Agora, veremos COMO essa integridade é mantida através de 4 atividades principais:





ITENS DE CONFIGURAÇÃO

ITENS DE CONFIGURAÇÃO DE SOFTWARE

- Durante o processo de desenvolvimento de um software, são produzidas grandes quantidades de itens de informações
- Alguns desses itens são selecionados de acordo com sua relevância e necessidade de controle tanto de versão quanto de mudança
- Esses itens selecionados são chamados **itens de configuração de software** e o conjunto dos mesmos compõe a **configuração de software**

O QUE DEVEMOS CONTROLAR?

- Identificar é "dar nome aos bois"
- Um IC não é apenas código!
- Exemplos de ICs em um projeto real:
 - Software: Código-fonte, binários, scripts de BD
 - Documentação: Requisitos, Plano de GCS, Manuais
 - Ambiente: Dockerfiles, arquivos de configuração (.yaml, .env)
 - Dados: Conjuntos de dados de teste ou modelos de IA treinados

ITENS DE CONFIGURAÇÃO DE SOFTWARE

É um **artefato de software** que precisa ser gerido a fim de entregar um produto ou serviço de software.

Itens que são selecionados de acordo com sua relevância e necessidade de controle tanto de versão quanto de mudança.

TODAS AS VERSÕES DE ARTEFATOS
AO REDOR DA ENGENHARIA DE
SOFTWARE, ENGLOBANDO NÃO
APENAS O CÓDIGO-FONTE, MAS
TAMBÉM A DOCUMENTAÇÃO,
MANUAIS, PÁGINAS DA WEB,
RELATÓRIOS, E OUTROS
ELEMENTOS ASSOCIADOS.

ITEM DE
CONFIGURAÇÃO

ITENS DE CONFIGURAÇÃO DE SOFTWARE

- Item de configuração (SCI – Software Configuration Item) é uma informação criada como parte do processo de engenharia de software
 - Em um caso extremo, poderia ser considerado uma única seção de uma grande especificação ou um caso de teste em um grande conjunto de testes
 - Em uma visão mais realista, um SCI é todo ou parte de um artefato (p. ex., um documento, um conjunto inteiro de casos de teste, um programa ou componente com nome, um ativo de conteúdo multimídia ou uma ferramenta de software)
 - Qualquer coisa associada a um projeto de software (projeto, código, dados de teste, documentos etc.) que tenha sido submetida ao controle de configuração
 - Os itens de configuração sempre têm um identificador único

EXEMPLOS DE ITENS DE CONFIGURAÇÃO (IC)

1. Especificação do Sistema
2. Plano de Projeto de Software
3. Especificação de Requisitos do Software
4. Manual Preliminar do Usuário
5. Especificação do Projeto
 - Descrição do Projeto de Dados
 - Descrição do Projeto Arquitetural
 - Descrições do Projeto Modular
 - Descrições do Projeto de Interface
 - Descrições de Objetos (se forem usadas técnicas orientadas a objetos)
6. Listagem do código-fonte
7. Planos, Procedimentos, Casos de Testes e Resultados Registrados
8. Manuais Operacionais e de Instalação
9. Programa Executável e Módulos Interligados
10. Descrição do Banco de Dados
 - Esquema e estrutura de arquivo
 - Conteúdo inicial
11. Manual do Usuário
12. Documentos de Manutenção
 - Relatórios de problemas de software
 - Solicitações de manutenção
 - Pedidos de mudança
13. Padrões e procedimentos para engenharia de software (templates, artefatos de processos)
14. Ferramentas de produção de software (editores, compiladores, CASE)

O QUE NÃO SERIA UM ITEM DE CONFIGURAÇÃO?

✓ Exemplo de Item de Configuração num sistema de vendas:

- O arquivo de código-fonte responsável por gerar a lista de vendas: esse arquivo impacta diretamente no comportamento do app e precisa ser controlado, versionado e rastreado

✗ Exemplo de algo que não é um Item de Configuração:

- Um rascunho informal feito à mão por um desenvolvedor com ideias para uma nova funcionalidade, ainda não aprovado ou incorporado ao projeto.
→ Não é um artefato oficial do projeto e não precisa de controle de versão ou rastreabilidade formal.

OS ATRIBUTOS DE UM IC

- Para ser gerenciado, um item precisa de metadados claros:
 - **Nome Único:** Ex: modulo_financeiro_v2
 - **Versão:** 1.0.4
 - **Responsável/Autor:** Quem criou ou mantém?
 - **Data de Criação/Modificação:** Rastreabilidade temporal.
 - **Relacionamentos:** De quais outros ICs este item depende?



VERSÃO
CONFIGURAÇÃO
BASELINE
RELEASE

VERSÃO

- (Dicionário) **Forma ou variação de algo** – Diferente apresentação ou edição de um texto, filme, software, etc.
- Se refere a uma identificação específica
 - Exemplo: "A nova versão do sistema foi lançada com melhorias."



The **Kanto** Pokédex in
Pokémon Red and Green



The **Kanto** Pokédex in
Generation I



The **Johto** Pokédex in
Generation II

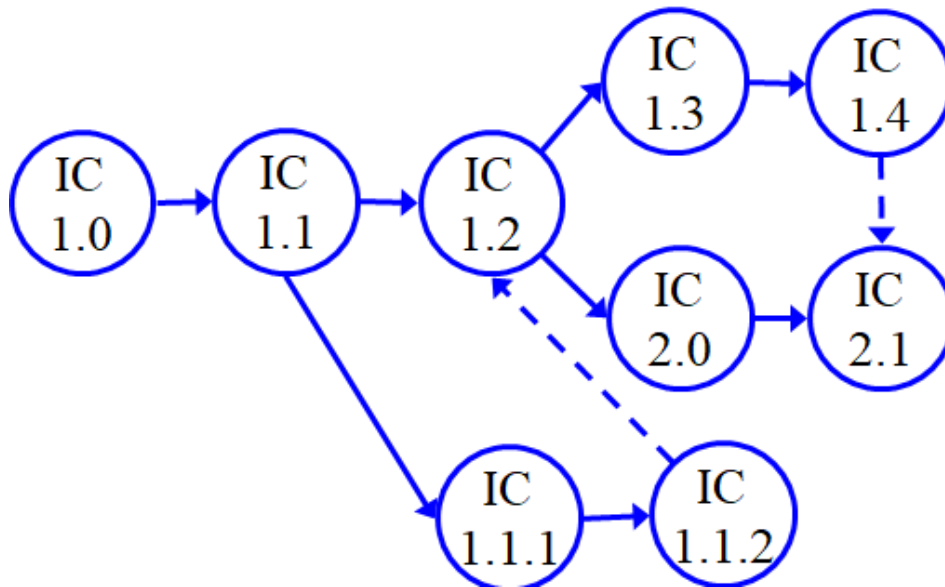


The **Hoenn** Pokédex in
Generation III



The **Kanto** Pokédex in
Generation III

VERSÃO



- Instâncias de um mesmo item de configuração que diferem entre si em algo (tipos: revisões e variantes)

VERSÕES: REVISÕES OU VARIANTES

➤ Revisões:

- Versões criadas para substituir versões anteriores seguindo uma linha temporal (e.g., em resposta a correção ou evolução)

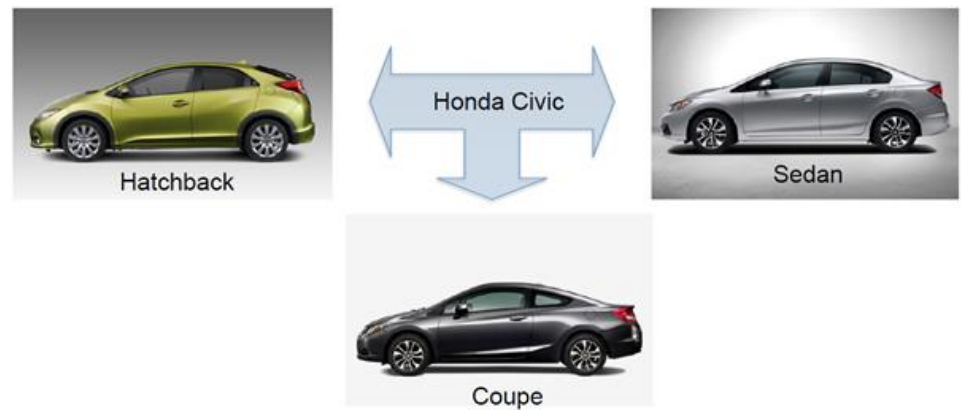
➤ Variantes:

- Versões coexistentes, projetadas para propósitos distintos (e.g., em resposta a diferentes arquiteturas de hardware ou sistemas operacionais)

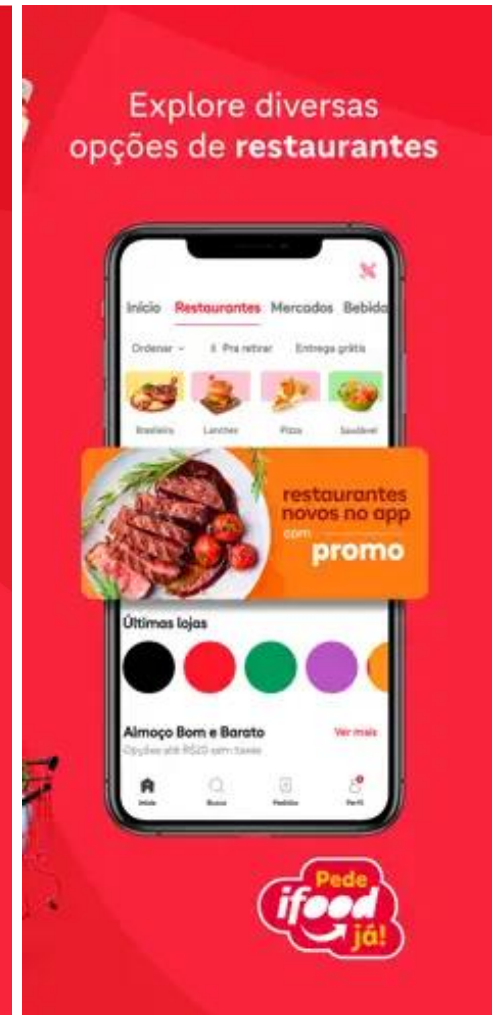
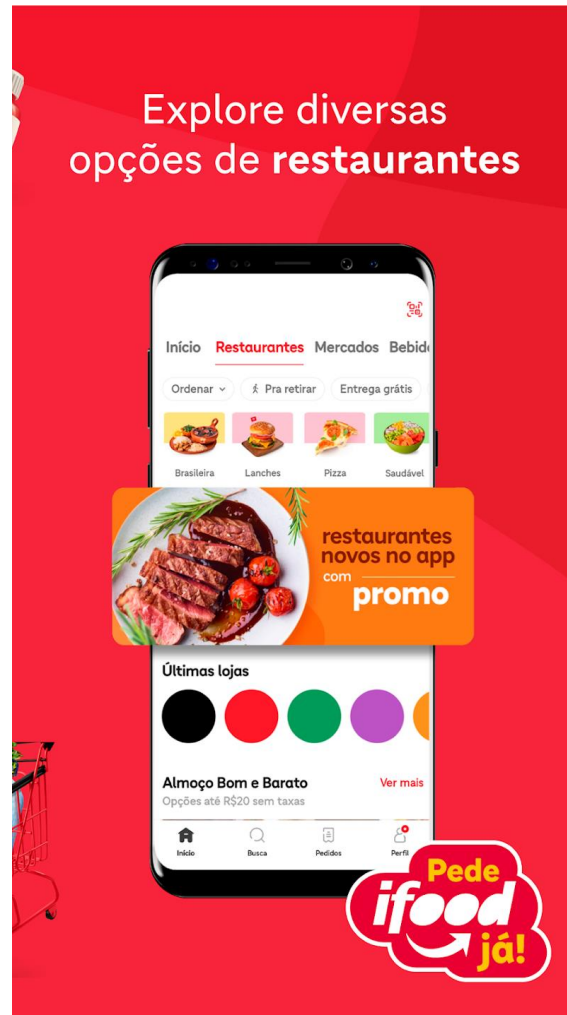
VERSÕES: REVISÕES OU VARIANTES



Variantes

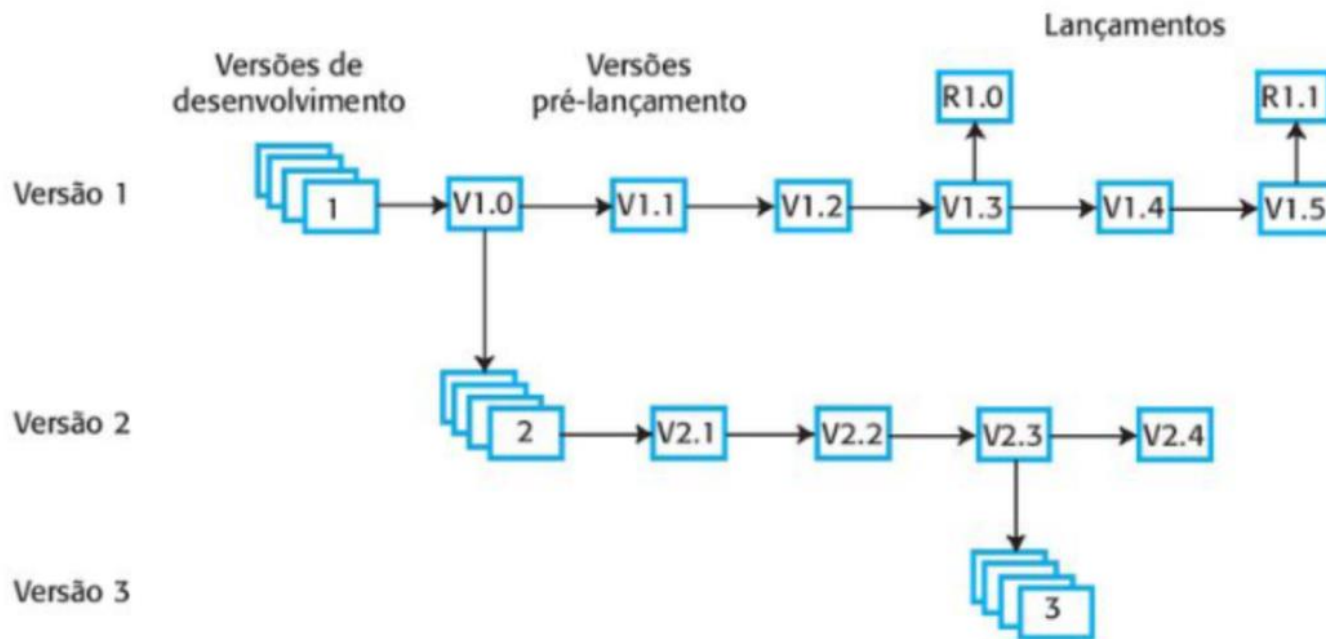


IFood (Android e iOS)
Variante?
E a versão web?



VERSÕES

FIGURA 25.2 Desenvolvimento de sistema multiversões.



Sommerville (2018)

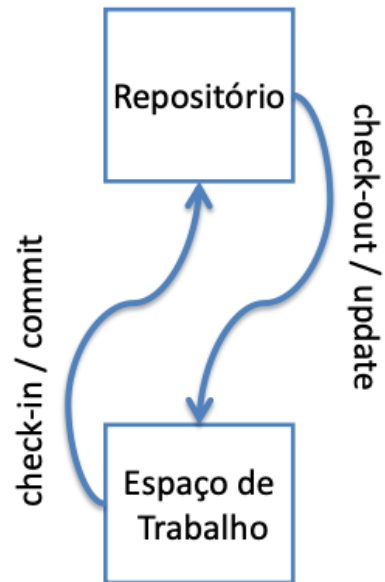
CONFIGURAÇÃO

- Um conjunto de versão de **Itens de Configuração (IC)**, onde existe somente uma versão selecionada para cada IC do conjunto
- Uma configuração pode ser vista como a versão de um IC composta de outros IC
- Exemplos:
 - Configuração do Sistema
 - Configuração do processo
 - Configuração do módulo X
 - Configuração dos requisitos do sistema
 - Configuração do código fonte

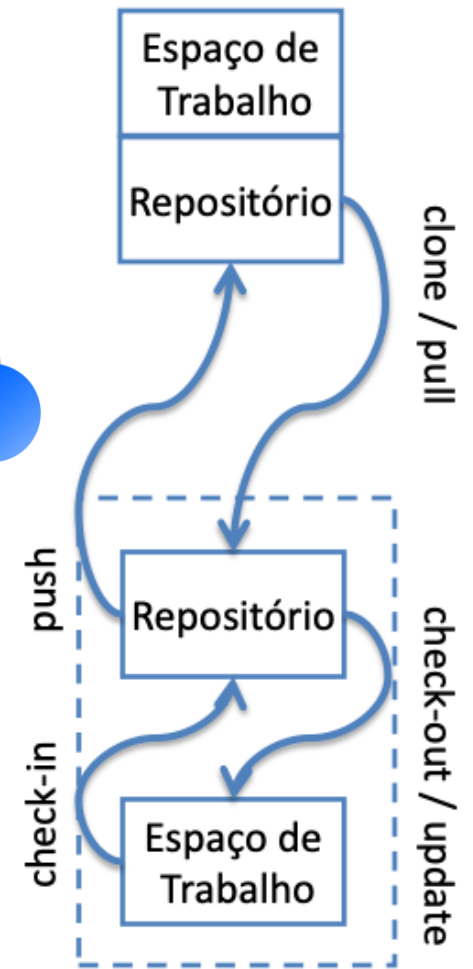
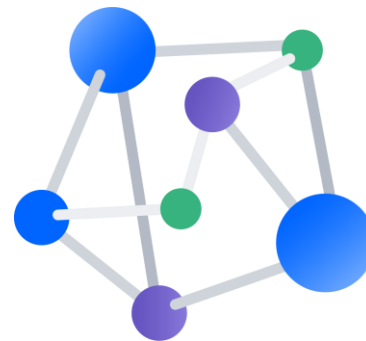
REPOSITÓRIO

- Local onde os ICs são armazenados
 - Armazena o histórico do projeto
 - Controle na entrada e saída de ICs
 - Poucos por projeto (normalmente, somente um)
- Utiliza diferentes mecanismos de armazenamento
 - Versionamento completo
 - Versionamento de diferenças (delta)
- Utiliza diferentes mecanismos de controle de concorrência;
 - Pessimista
 - Otimista
 - Misto
- Permite a geração de diferentes relatórios
 - Por item de configuração
 - Por modificação

TOPOLOGIA DE UM REPOSITÓRIO

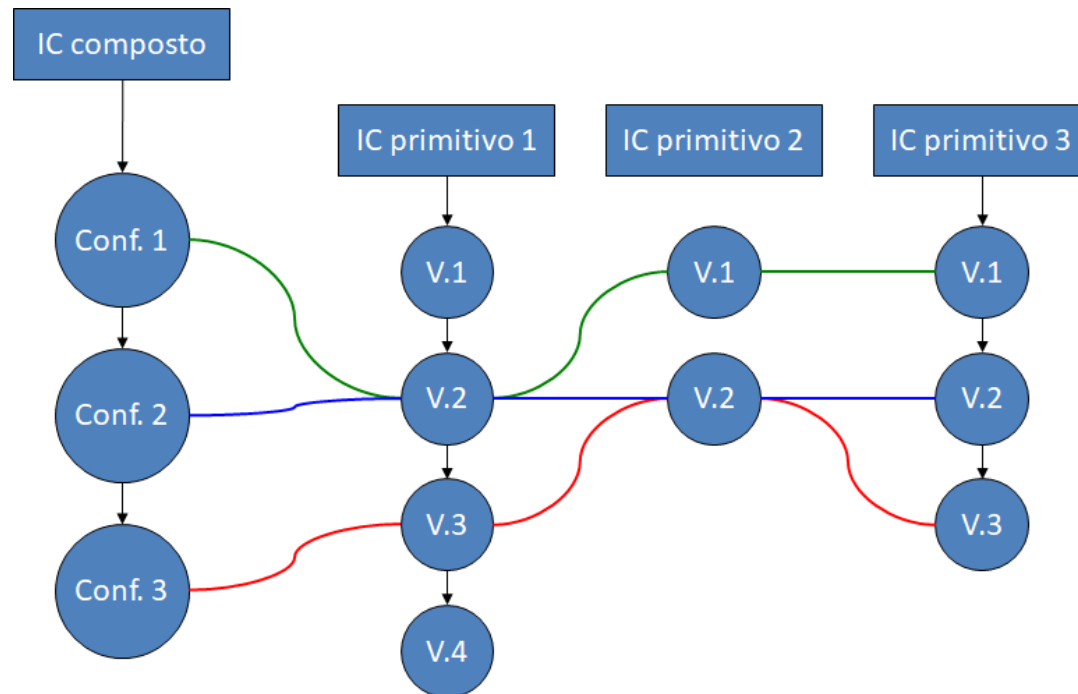


Centralizado



Distribuído

CONFIGURAÇÃO X VERSÃO



BASELINE

- Uma *Baseline* é um conjunto de ICs que foi formalmente revisado e aprovado em um determinado momento

- Analogia:
 - É como uma "foto" oficial do projeto que serve como base para o desenvolvimento futuro

- Características:
 - Só pode ser alterada através de procedimentos formais de controle de mudanças
 - Garante que todos os membros da equipe estão trabalhando sobre a mesma base estável

BASELINE

- Configuração revisada e aprovada que serve como base para uma próxima etapa de engenharia e que somente pode ser modificada via processo formal de GCS
- São estabelecidas ao final de cada fase de desenvolvimento: análise (functional), projeto (allocated) e implementação (product)
- Momento de criar: balanceamento entre controle e burocracia



Teremos aula específica
para baselines. Incluindo
os níveis de controle

BASELINE

Versão específica de um artefato ou conjunto de artefatos que foi formalmente aprovada e serve como base para futuras mudanças.

Representa um ponto de referência estável durante o ciclo de vida do software.



QUANDO MARCAMOS UMA BASELINE?

- Baseline de Requisitos: Quando o cliente aprova o que deve ser feito
- Baseline de Design: Quando a arquitetura é definida
- Baseline de Produto (Release): Quando o software vai para produção

Se algo der errado na versão 2.0, a Baseline 1.0 é o que nos permite reconstruir tudo com segurança.

EXEMPLO: BASELINE DE REQUISITOS FUNCIONAIS V1.0

- Conjunto aprovado de requisitos que define as funcionalidades iniciais de um app, como:
 - Criar listas de equipamentos
 - Salvar e editar listas
 - Consultar peso total da mochila
- A partir dessa baseline, o desenvolvimento inicial do app é iniciado, e qualquer alteração futura nos requisitos passa por controle formal

EXEMPLO: BASELINE DE CÓDIGO V1.0

- Versão estável e testada do código que foi aprovada para ser lançada na Play Store
- Inclui:
 - Código-fonte.
 - Scripts de build.
 - Assets (ícones, imagens, etc.)
- Serve como referência caso seja necessário corrigir bugs sem afetar o desenvolvimento de novas funcionalidades (ex: criar um branch de hotfix a partir dessa baseline)

EXEMPLO

- Um time de engenharia está trabalhando em um projeto e tenha completado a fase de levantamento de requisitos
- Neste ponto, os requisitos foram revisados, validados e aprovados pelos stakeholders. A versão final desses requisitos é então definida como a "baseline de requisitos"
- Qualquer mudança subsequente nos requisitos deve ser cuidadosamente gerenciada e passar por um processo formal de controle de mudanças para garantir que não afete a baseline estabelecida

CONSTRUÇÃO (*BUILDING*)

- Processo de compilação do sistema a partir dos itens fonte para uma configuração alvo
- Utiliza arquivo de comandos que descreve como deve ocorrer a construção
 - Exemplo: makefile, build.xml, pom.xml;
- Os arquivos de comandos também devem ser considerados itens de configuração

BUILD

- Uma versão compilada e executável de um software
- Transformação do código-fonte em artefatos executáveis, como binários, arquivos de instalação ou pacotes distribuíveis
- Pode incluir compilação de código, criação de artefatos, execução de testes automatizados e empacotamento de recursos necessários

```
Creating an optimized production build...  
Compiled successfully.
```

```
File sizes after gzip:
```

```
528.01 KB build/static/js/main.9229ae1b.js  
12.39 KB build/static/css/main.31370571.css
```

```
The bundle size is significantly larger than recommended.  
Consider reducing it with code splitting: https://goo.gl/hN5waj.  
You can also analyze the project dependencies: https://goo.gl/4cyMnA.
```

```
The project was built assuming it is hosted at the server.  
You can control this with the homepage field in your package.json.  
For example, add this to build it for GitHub Pages:
```

```
"homepage" : "http://myname.github.io/myapp",
```

```
The build folder is ready to be deployed.  
You may serve it with a static server:
```

```
npm install -g serve  
serve -s build
```

```
Find out more about deployment here:
```

```
http://bit.ly/2vY88Kr
```


LIBERAÇÃO (*RELEASE*)

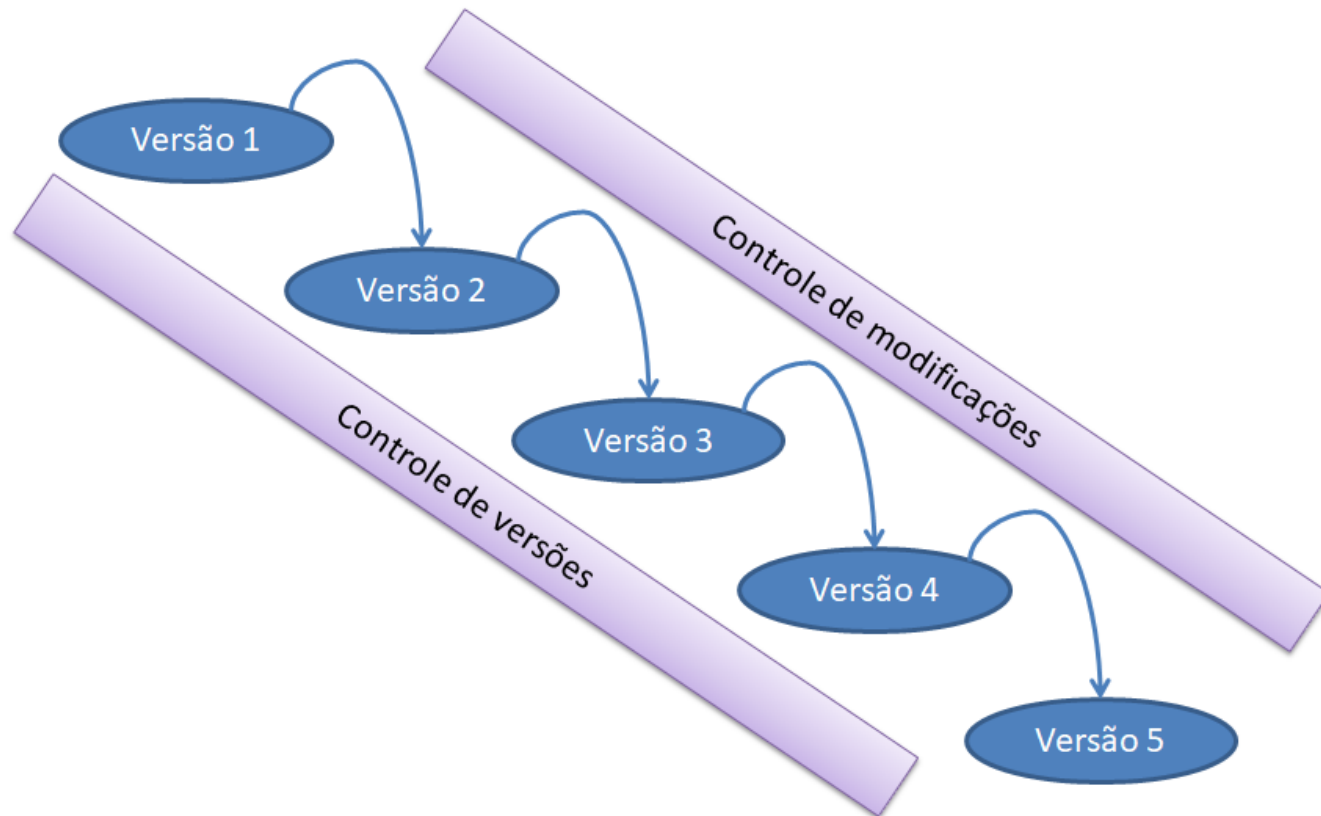
- **Substantivo:** Versão disponibilizada para um propósito específico
- **Verbo:** Notificação formal e distribuição de uma versão aprovada
- **Importante**
 - Toda liberação é uma versão
 - Nem toda versão é uma liberação
- Em alguns casos liberações podem ser desenvolvidas em paralelo (time to market)
- **Exemplos**
 - Liberação para testes de sistema
 - Liberação para homologação
 - Liberação para entrega ao cliente

E QUAL A DIFERENÇA ENTRE RELEASE E BUILD?

- Uma "build" refere-se à compilação de código-fonte em um estado executável ou em um pacote que pode ser distribuído
- "Release" é uma versão específica do software que é disponibilizada para os usuários finais, após testes e possíveis correções de bugs
- Portanto, uma build pode resultar em várias releases, dependendo do ciclo de desenvolvimento do software



SISTEMA DE GERÊNCIA DE CONFIGURAÇÃO



© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved.
© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved.
© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved.





RELATÓRIOS E AUDITORIA

RELATÓRIOS DE STATUS

- Onde estamos agora?
- Esta atividade responde a perguntas críticas dos gestores:
 - "Quais mudanças foram feitas desde a última Baseline?"
 - "Qual versão do manual acompanha a versão 2.1 do código?"
 - "Quantas solicitações de mudança (CRs) estão pendentes?"
- Objetivo: Fornecer visibilidade e rastreabilidade total sobre a evolução do sistema.

RELATÓRIOS DE STATUS

- É uma comunicação regular que fornece informações sobre o andamento, progresso e situação de um projeto de software
- Ele é usado para manter as partes interessadas informadas sobre o estado do projeto

RELATÓRIOS DE STATUS

➤ Exemplo:

- Informações sobre as versões dos artefatos, quais mudanças foram implementadas, quem fez as mudanças, quando foram feitas, e qualquer outra informação relevante sobre a configuração do software.
-
- Esse tipo de relatório é essencial para garantir a rastreabilidade e o controle adequado das mudanças no software ao longo do tempo.

AUDITORIA DE GERÊNCIA DE CONFIGURAÇÃO

- É um processo formal de revisão e avaliação dos processos, procedimentos, práticas e registros relacionados à gerência de configuração de software.
- O objetivo é garantir que as políticas e padrões estabelecidos estejam sendo seguidos corretamente e que o ambiente de gerência de configuração esteja funcionando de maneira eficaz

AUDITORIA DE GERÊNCIA DE CONFIGURAÇÃO

- Garantir a conformidade com os padrões e práticas estabelecidos
- Identificar áreas de melhoria nos processos de gerência de configuração
- Assegurar a integridade e rastreabilidade das configurações de software

- Aspectos Avaliados
 - Conformidade com os processos estabelecidos
 - Integridade e rastreabilidade das configurações
 - Adequação da documentação de configuração
 - Eficácia dos controles de versão e mudança

AUDITORIA DE GERÊNCIA DE CONFIGURAÇÃO

- Garante que o que foi planejado é o que foi entregue
- Auditoria de conformidade de configuração:
 - Um auditor revisa os artefatos de configuração de software, como planos de configuração, baselines, registros de mudanças e documentos de controle de versão, para garantir que eles estejam em conformidade com os padrões e políticas estabelecidos pela organização.
 - Exemplo: auditor verifica se todas as mudanças foram devidamente documentadas, se as baselines foram estabelecidas e mantidas corretamente, e se os procedimentos de controle de versão foram seguidos adequadamente

CONCLUSÃO E PRÓXIMOS PASSOS

➤ Resumo:

- GCS é um ciclo: **Identificamos** os itens, marcamos **Baselines**, acompanhamos o **Status** e **Auditamos** a qualidade
- Atividade: "Acesse o AVA e realize a Atividade Prática: Identifique 5 ICs de um projeto seu (que não sejam código) e seus atributos."
- Próximo Módulo: Módulo 3 – O Plano de GCS (Colocando tudo no papel conforme a IEEE 828).